

UNIT-4.

MEMORY MANAGEMENT.

The main purpose computer system is to execute Programs. These programs with their accessing data must reside in the main memory at the time of their execution.

Since main memory is usually too small to accommodate all the data and programs permanently, the computer system must provide secondary storage to back up the main memory.

ADDRESS BINDING MECHANISM.

usually a program resides on disk as a binary executable file. The program must be brought in main memory and placed within a process for its execution. The collection of process as on the disk waiting to be brought into memory forms the input queue.

The procedure is to select one of the process from this queue and load it into main memory.

Addresses in the source program are generally symbolic.

A compiler will bind these address to relocatable add. The linkage Editor or loader will bind these Add to absolute add.

The binding of data and program to memory addresses can be done at any of the following step -

- (i) Compile Time $\Rightarrow$ Binding at compile time generates absolute addresses where a prior knowledge is required that where a process resides in the memory. After sometime if the starting location of a process in memory is changed, then the entire process must be recompiled to generate the absolute address again.
- (ii) Load Time - At compile time if it is not known that where a process will reside in memory then the compiler must generate relocatable Address. In this case final binding is delayed until load time. If the starting Address is changed then we need only to reload the user code to incorporate this changed value.

(iii) Execution Time -! This method permits moving a process from one memory segment to another during run time.

RELOCATION -!

Relocation is a method of shifting a user program from one memory location to another.

Types of Relocation -

- 1 $\Rightarrow$ Static Relocation
- 2 $\Rightarrow$ Dynamic Relocation

Static Relocation -! Static Relocation takes place at load time and fixed once the program is relocated.

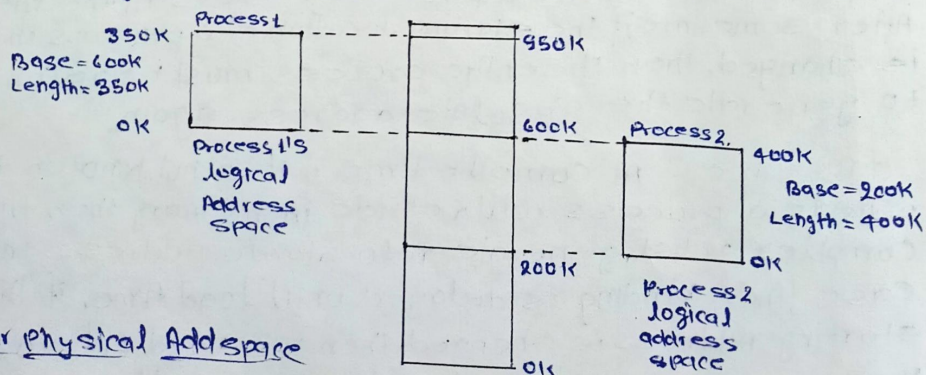
Dynamic Relocation -! takes place at run time.

Logical & Physical Address Space -!

Logical Address are generated by CPU where as Physical Address the Address which refers in main memory obtain from memory address register are called physical address.

Set of all logical addresses generated by a program is called logical address space and set of all physical addresses which corresponds to these logical addresses is called physical Address Space.

Note -! physical addresses are used in system mode while logical Addresses are used in user mode.



computer physical Address space

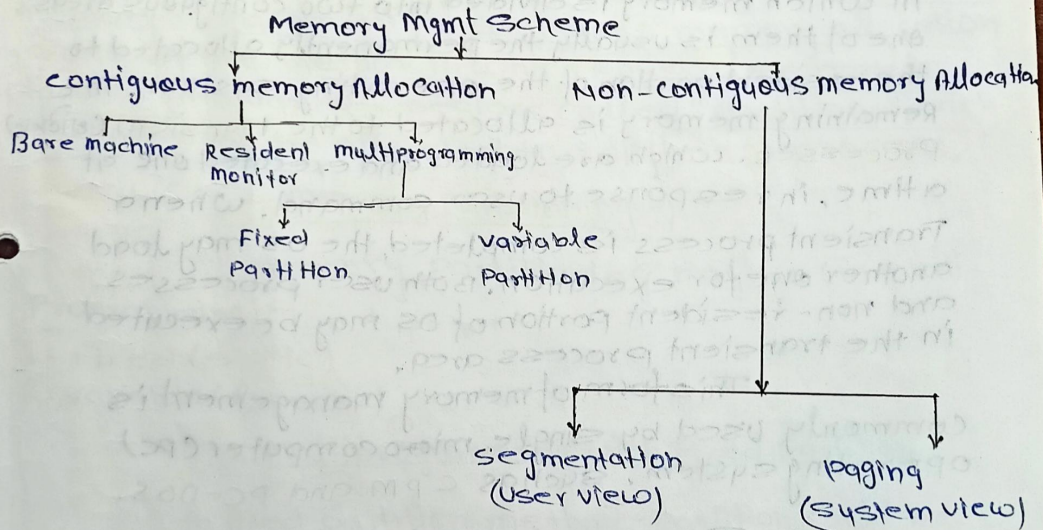
MEMORY MANAGEMENT SCHEMES

memory management schemes are broadly divided into following categories-

1. contiguous memory Allocation -! Contiguous allocation means that each logical object is placed in a set of memory locations with strictly consecutive addresses.

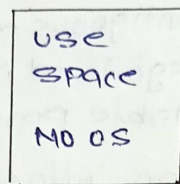
Bare machine, Resident monitor, multi programming are further division of contiguous allocation. while multiprogramming is categorised into two parts - Fixed partition and variable partition.

2. Non Contiguous memory Allocation - Non-contiguous allocation implies that a single logical object may be place in non-consecutive set of memory location. Paging (system view) and segmentation (user view) are two mechanisms that are used to manage non-contiguous memory Allocation.



Contiguous memory Allocation -

1. Bare machine (Single user) - It is one of the most simplest memory management method in which user has complete control over the memory while in other methods operating system has complete control over memory.



100K

OK

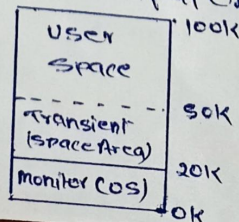
Bare machine

2. Resident monitor (Single process Monitor) - It is also a simplest memory management method in which memory is divided into two contiguous areas, one of them is usually permanently allocated to the resident portion of the operating system. The remaining memory is allocated to the transient (User Processes), which are loaded and executed one at a time, in response to user command. When a transient process is completed, the OS may load another one for execution. Both user processes and non-resident portion of OS may be executed in the transient process area.

This form of memory management is commonly used by single microcomputer (PC) operating system, such as CPM and PC-DOS.

The name is derived from a program which is always present in the computer's memory thus being "resident."

Resident
monitor →



± MULTIPROGRAMMING ~~with fixed partition~~ -1.

To improve the performance of the computer system, The concept of Multiprogramming was introduced, which states that "more than one processes are accessed at a single instant of time". But only one process is executed by CPU at a time in a uniprocessor system and at that time particular time other processes may be accessed by I/O processor which process is executed and in which order is decided by CPU Scheduler.

To access more than one processes at a time, they must reside in our memory. For this, we should know that how more than one processes can be allocated in our memory.

Various memory management policies are developed to provide the solution of this problem in a multiprogramming environment. Partitioned memory management for contiguous memory allocation is one of that method in which physical memory is divided into various sectors called "partitions", each of which may be allocated to a different process.

± Memory Partitioning -1

memory partitioning may be (in multi-Programming) Two types +

- 1 = Fixed (static)
- 2 = variable (dynamic)

± multiprogramming with Fixed Partition -1

in fixed partition method, partition could be of different sizes but once decided at the time of system generation they could not be changed i.e. fixed.

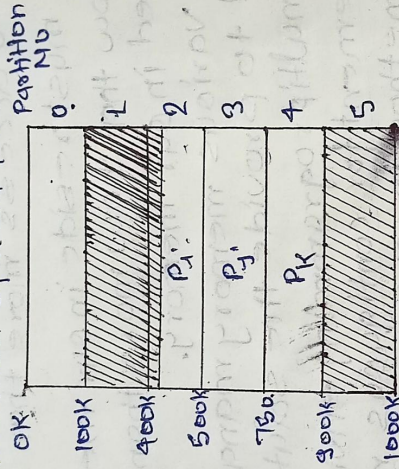
This method could be used with swapping or relocation.

The number and sizes of individual partitions are determined at the time of system generation.

→ Degree of multiprogramming -

The numbers of processes reside in main memory at a time is called degree of multiprogramming.

Example - As shown in the fig. A partitioned memory depicted in fig. out of six partitions, one is assumed to be occupied by the resident portion of OS and three partitions are allocated by the processes P_i , P_j and P_k . The remaining two shaded partitions are free and available for allocation of next process.



Static partition

Once partition are defined, an OS needs to keep track of their status. Such as free or in use, for allocation purpose. Current partitions status and attributes are collected in a data structure called partition description Table (PDT) as shown in the fig.

Partition No	Partition Base	Partition Size	Partition Status
0	OK	100K	Allocated
1	100K	300K	free
2	400K	100K	Allocated
3	500K	250K	Allocated
4	750K	150K	Allocated
5	900K	100K	free

Partition Description Table (PDT).

Problems occurs in fixed partition:-

in static / Fixed partition, Two major problems may occur:-

- (i) How to select a specific partition for a given process
- (ii) what is done when no suitable partition is available for allocation.

The first problem can be solved in several ways, for which first fit and the best fit are probably the common strategies.

First fit Approach:- The first fit Approach basically consists of all allocating the first free partition large enough to accommodate the process being created.

Best fit Approach:- The best fit approach requires that the operating system allocate the smallest free partition that meet the requirement of the process.

Note → The first fit terminates upon finding the first such partition. The best fit must process all qualifying PPT entries to identify the tightest fit.

First fit is faster to execute.

Best fit is higher utilization of memory.

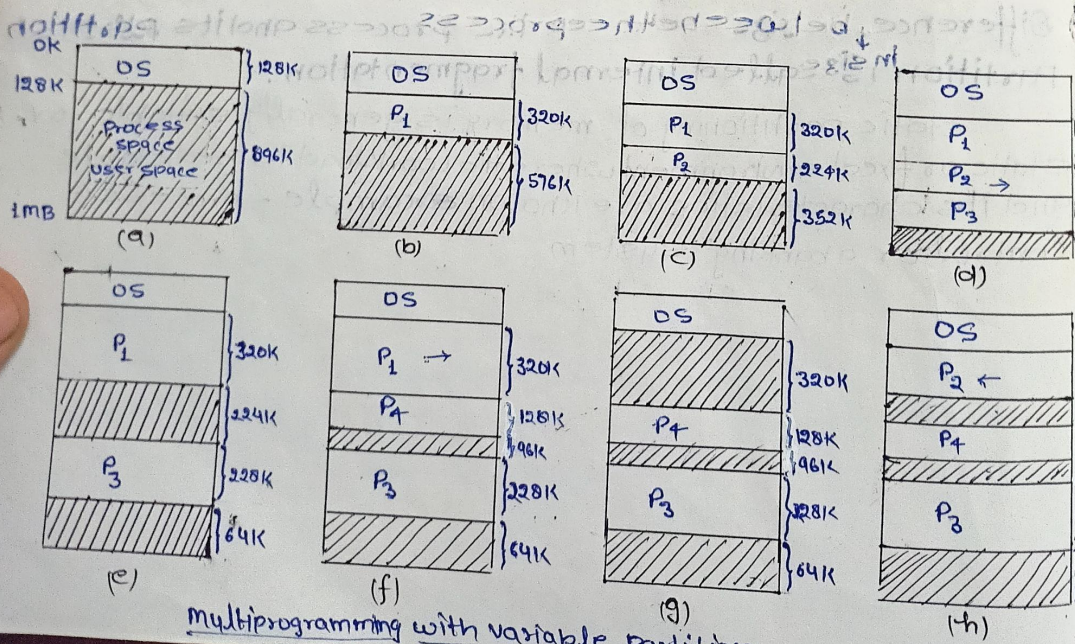
Internal fragmentation:-

Difference in size between the process and its allocated partition is called internal fragmentation.

static partitioning of memory is generally suitable for static or fixed environment where the workload is predictable and its characteristics are known Example - process control or a banking system.

Multiprogramming with variable partitioning

in dynamic partitioning, the partitions used are of variable length and number. when a process is brought into memory, it is allocated exactly to that amount of memory that it needs, not more than that it requires. An example shown in figure uses 1MB memory for dynamic partitioning. The first three processes P_1, P_2, P_3 are loaded starting where OS ends and occupy just enough space for each process (figure, b, c, d). This leaves a small "hole" at the end of memory that is very small for the fourth process P_4 . Then the operating system must swap out any of the three process whose execution is completed or the process which is not required presently. according to P_2 is swapped out (figure-e) which leaves sufficient space to load the P_4 (fig. f). Since process P_4 is smaller than P_2 so another hole is created. After sometime there may be a situation when no process is present in ready queue and P_2 wants to be executed from waiting queue. Then the OS swaps process P_1 out and process P_2 back (fig. h).



multiprogramming with variable partition

As shown in but after some and memory k This phenomenon compaction fragmentation the process scattered placed tog contiguous compaction is shown in A block created in u process co

Here are the area of mem) First fit - beginning of first fit se Best fit - is large e unless it is Worst fit - we must s Sorted by s

partitioning -

processes used
process is
to that amount
it requires,
ory for
s P_1, P_2, P_3
occupy just
). This leaves
very small
system
whose
is not
ed out
to load
than P_2
e there
sent in
om
 P_1 out

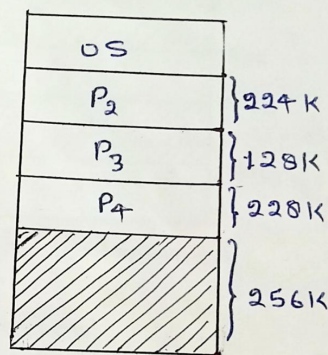
External Fragmentation -

As shown in the last Example, this method is started well but after some time it results a lot of small holes in memory, and memory becomes more and more fragmented as time goes on. This phenomenon is called External fragmentation.

compaction - One technique for overcoming external fragmentation is compaction, which means to move the processes in the memory in such a way that the scattered pieces of unused memory (holes) can be placed together, so that any other process demanding contiguous memory for it can use.

Compaction method or Technique is shown in the fig.

A block of 256K memory is created in which any other new process can be accommodated.



compaction

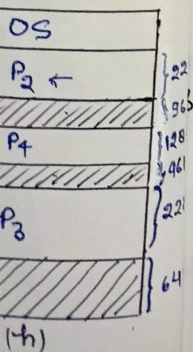
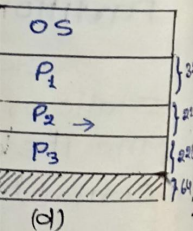
Selection of free area of memory -

There are three algorithms used for selection of a free area of memory to create a partition -

1) First fit - In first fit we allocate the first hole that is big enough. Searching can be started either at the beginning of the set of holes (First fit) or where the previous first fit search ended (Next Fit), (First hole large enough)

2) Best fit - In best fit we allocate the smallest hole that is large enough. We must search the whole (full) list unless it is sorted by size. (Least size of unused)

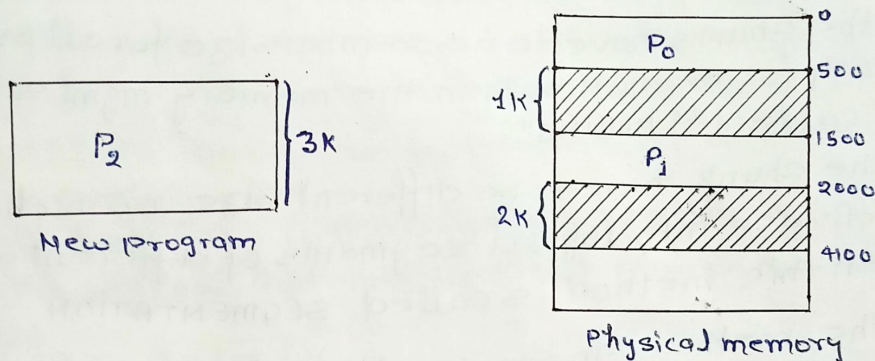
3) Worst fit - In worst fit we allocate the largest hole. Again we must search the entire (whole) list, unless it is sorted by size. (largest size of unused)



← Non-contiguous memory Allocation -!

Fragmentation problem that arises in the contiguous memory allocation. So the solution of the fragmentation problem, a better method to overcome the fragmentation problem was evolved which makes our logical address space - Non-contiguous in nature.

Example -! considering the fig, in which before applying compaction, there are holes of sizes 1K and 2K.



Fragmentation Problem revisited

If a New Process P_2 of size $3K$ wants to be executed then its execution is not possible without compaction as stated in above mechanism of memory management, But since compaction takes a lot of time.

An alternative approach to overcome the fragmentation problem is made as -

We divide the size of new process P_2 into two chunks of 1K and 2K to be able to load them into two holes at different places.

Note -! We change the process contiguous to non-contiguous.

In the above Example we have two base registers for two different chunks. Each of which will have the value of the memory address of the beginning

of that chunk. For Example as shown in the previous fig. Process P_2 will be loaded in two chunks starting at the base address 500 and 2000 respectively. So base registers will have value 500 and 2000. To arrive at final physical address by address translation the respective base register will be added to the original address.

What will be the size of chunk

- (1) if the chunks have to be same size for all processes ready for execution then the memory mgmt scheme is called "PAGING"
- (2) if the chunk have to be different size in which process is divided into logical segments of different sizes then this method is called SEGMENTATION.
- (3) if the method can work with only some chunks in main memory and the remaining on the disk which can be brought into main memory only when it is required, then the system is called VIRTUAL MEMORY MANAGEMENT SYSTEM.
4. There are other Hybrid method such as segmented paged method, in which each process image is divided into a number of segments of different size and each segment is divided into many same size pages.

Note

Physical
Page J
two p

Paging (System View)

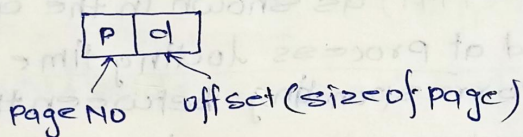
Paging is a non-contiguous memory management schemes. This scheme permits the physical address space of a process to be non-contiguous.

The physical memory is conceptually divided into a number of fixed-size blocks, called frames and process is also splitted into fixed size block called pages. When a process is to be executed, its pages are loaded into any available memory frames.

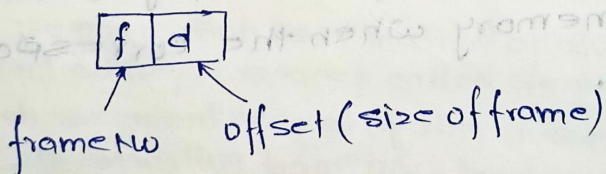
The size of the frame is always same as the size of the page for a particular hardware.

Addresses:- There are two types of address are used in paging.

1. Logical Address - Logical Address is always generated by CPU. Logical Add has two part Page no and size of page.



2. Physical Add:- Physical Add show the Add of frame in main memory, Physical Add also has two parts frame No and size of frame

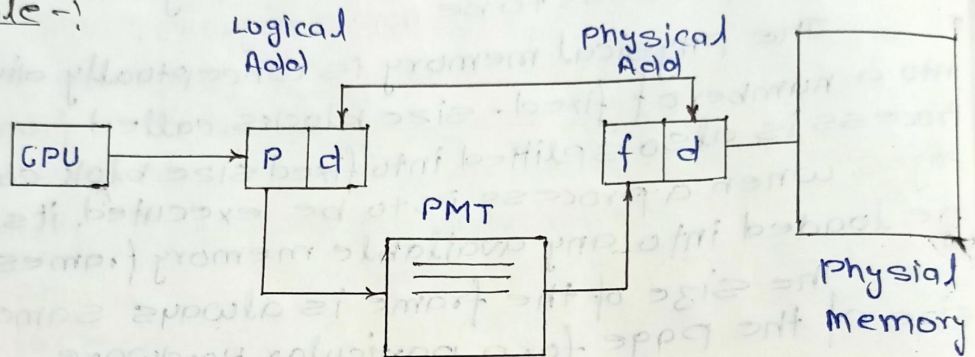


Note → size of frame is always same as the size of page.

The mapping of logical address to Physical address in paging system is performed at page level. Particular logical Add is divided into two parts the page NO (P) and the offset (d) within

that page. Since page (p) and frame (f) have identical size, so offset within each are identical and not need to mapped.

Example -1



Paging System

In paging system, address translation is performed with the help of a mapping table called as Page-map-table (PMT) as shown in the above fig. PMT is constructed at process loading time in order to establish the correspondence between the virtual and physical addresses. In PMT there is one entry for each page of process indexed with the page no. The value of each entry is the number of frames (composed of high order, page level bits) in the physical memory when the corresponding page is placed.

identical
and not need

Segmentation (user view)

Segmentation is another technique for non-contiguous storage allocation. It is different from paging as pages are physical in nature and hence are of fixed size, whereas segments are logical divisions of a program and hence are of variable size.

It is a memory management schemes that supports the user view of memory rather than system view of memory as in paging. in which we concerned on division of physical memory into fixed size blocks called frames. In segmentation we divide the logical address space into different segments.

Each segments has a name and length which is loaded into physical memory as it is. for simplicity the segments are referred by a segment NO, rather than segmen name.

Logical address consist of two tuples

$\langle \text{segment-name, offset} \rangle$

Address is considered as two dimensional, The size of a segment varies according to the data stored in it or the nature of operation performed on that segment.

So Segmentation is a method of dividing memory into independent address spaces called Segment. Each

Each segment consist of linear sequence (0, 1, 2, 3, ...) of address starting from "0" to "maximum" value depending upon the size of segment.

Compiler itself has to generate the two-dimensional address (s, d) i.e (segment no, offset)

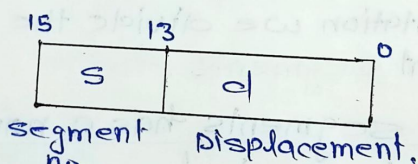
Segment No	0	1	2	3
Limit Address	0-999	1000-1699	1700-2499	2500-3399

virtual address segmentation

This is different from the paging system in which a single-dimensional virtual add and two-dimensional add would be exactly same in the binary form as the size of page is an integral power of 2.

in Segmentation it is not possible as the size of segment is unpredictable. So we need to express the add in 2-d form,

So virtual add space is divided into two parts in which high order unit refer to "s" i.e segment number and lower order units refer to "d" i.e displacement (limit value).



virtual or logical address space

VIRTUAL MEMORY

Virtual memory is a technique that allows the execution of processes that may not be completely in memory.

It is a virtual resource of a computer.

It is an illusion that a computer system processes more memory than it is actually having.

Virtual memory allows execution of partially loaded processes. As a consequence, the sum of virtual address space of active processes in virtual-memory system can exceed the capacity of available physical memory provided that the physical memory is large enough to hold a minimum amount of the address space of each active process.

This is accomplished by maintaining an image of the entire virtual address space of a process on secondary storage and by bringing its section into main memory when needed. The choice of which sections to bring in, when to bring them in, and where to place them is made by the operating system. These decisions are influenced by the demands of the active processes and by overall resource availability.

IMPLEMENTATION OF VIRTUAL MEMORY

Virtual Memory is implemented by demand paging.

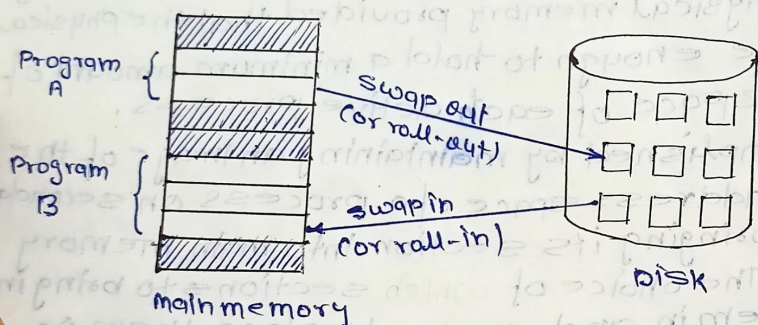
DEMAND PAGING

In demand paging, a page is brought into the memory for its execution only when it is demanded, otherwise it is remained in backing storage (disk).

The name seems to have been derived from demand feeding - A policy used for feeding the baby by mothers in which the food is given to the baby only when he cries for it.

This method is the combination of Paging and swapping method. The main requirement of this method is that the complete program should be present in the backing storage (disk) in the form of pages. Swapping is used to swap the contents of the program from disk to main memory when it is required.

In this method a "lazy swapper" is used, which swaps the pages when it is needed. Swapping requires removing the unused contents from main memory to disk and loading the required content of program from disk to main memory.



Swapping with paging

Advantages:-

1. Reduced memory requirement
2. Swap time also reduced
3. Increase degree of multi programming

Disadvantages:

Using this method swapping is done using hit and trial method. For example - if a program is having 5 pages 0, 1, 2, 3, 4 and out of these only page 0, 3, 4 are loaded into memory at the time of execution state. If the program tries to access the address of page NO-2, then there will be an error, because this page is not present in the main memory since it was not loaded. This error is called as Page fault.

Page Replacement Algorithm,

There are several page Replacement Algorithms given below -

1. First-in-First Out (FIFO) Replacement Algorithm,

This algorithm states that the page which comes first in the main memory, that page will be selected for swap out from main memory.

- (1) very simple in implement
- (2) oldest page is selected for replacement
- (3) Performance is not always good

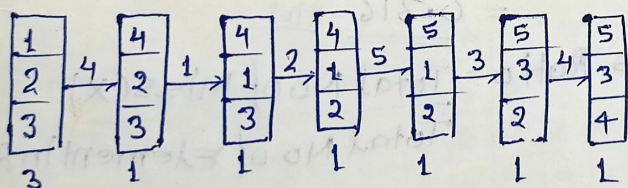
Belady's Anomaly - For some page replacement Algorithm, page fault may increase as the No of allocated frames $\uparrow$ (increase).

Prob - calculate no of page fault for following Ref-strings. 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Soln case-1.

Assume the No of frames = 3

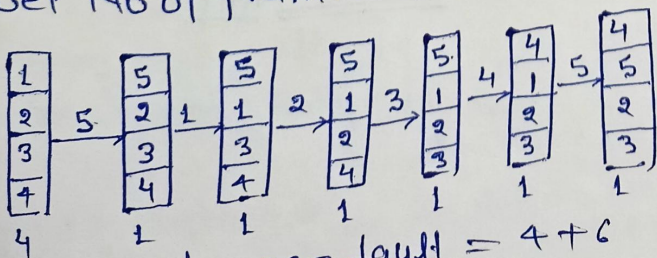
Given Ref string = 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5.



total No of page fault = 3 + 6
= 9.

case-2.

Let No of frame = 4



total No of page fault = 4 + 6
= 10

Belady's Anomaly

$\uparrow$ Frame = $\uparrow$ Page fault

3 = 9

4 = 10

Prob-2. Given Ref string.

3, 2, 1, 3, 4, 1, 6, 2, 4, 3, 4, 2, 1, 4, 5, 2, 1, 3, 4

Calculate total No of page fault.

calculate the Hit Ratio

calculate the miss Ratio,

Soln.

oldest page selected as victim
page

Frame Pages	3	2	1	3	4	1	6	2	4	3	4	2	1	4	5	2	1	3	4
f ₁	3	3	3	3	4	4	4	4	4	3	3	3	3	3	5	5	5	5	4
f ₂		2	2	2	2	2	6	6	6	6	4	4	4	4	4	2	2	2	2
f ₃			1	1	1	1	1	2	2	2	2	2	2	1	1	1	1	3	3
H(V) M(X)	X	X	X	✓	X	✓	X	X	✓	X	X	✓	X	✓	X	X	✓	X	X

Total No of Page fault (miss 'x') = 13. Ans

Hit Ratio = $\frac{\text{Total No of Hit (✓)}}{\text{Total No of Element in Ref string}}$

$$= \frac{6}{19}$$

$$= 0.316 \text{ Ans}$$

Miss Ratio = $\frac{\text{Total No of Miss (X)}}{\text{Total No of Element in Ref string}}$

$$= \frac{13}{19}$$

$$= 0.684 \text{ Ans}$$

Best method.
to
solve.

Prob-2

Page/Fram

f₁

f₂

f₃

Hit (✓)

miss (X)

2- Least Recently Used (LRU) Algo (PAST)

in this algorithm, the page that has been not used for longest period of time is selected for replacement.

It is like optimal page replacement algo looking backward in time.

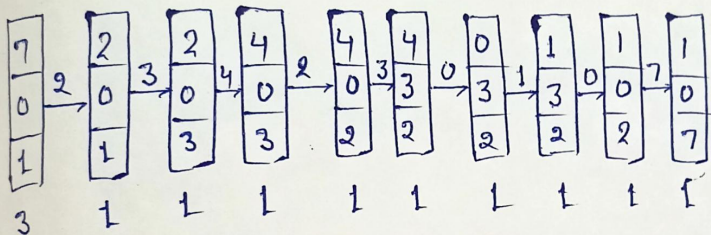
ded as victim
page

5	2	1	3	4
5	5	5	5	4
4	2	2	2	2
1	1	1	3	3
X	X	✓	X	X

Job: calculate the No of page fault for following Reference strings.

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Let No of frame = 3.



Ans

✓)

in Ref string

Total No of page fault = 3 + 9
= 12

Job 2

ss(X)

ement in Ref string

Ref string	3	2	1	3	4	1	6	2	4	3	4	2	1	4	5	2	1	3	4
% Frame	3	2	1	3	4	1	6	2	4	3	4	2	1	4	5	2	1	3	4
f1	3	3	3	3	3	3	6	6	6	3	3	3	1	1	1	2	2	2	4
f2		2	2	2	4	4	4	2	2	2	2	2	2	2	5	5	3	3	
f3			1	1	1	1	1	1	4	4	4	4	4	4	4	4	1	1	1
(v)	X	X	X	✓	X	✓	X	X	X	X	✓	✓	X	✓	X	X	X	X	X
miss(x)																			

Note → Replace a page which is not in use for a long time.

total Page fault = 14

Total miss Ratio = $\frac{14}{19}$ Ans

Hit Ratio = $\frac{5}{19}$ or Hit Ratio = $(1 - \text{miss Ratio})$
= $1 - \frac{14}{19} = \frac{5}{19}$ Ans

comparision of FIFO/LRU/optimal

According to problem 2.

Algo	Page fault	Hit Ratio
FIFO	13	6/19
LRU	14	5/19
Optimal	10	9/19

after optimal
Page Replacement

for all ϵ

which u
i.e futu

⇒ lowes

⇒ Replace

Period

$\Rightarrow$ It is v

Prob:-

refere

Soln

7, 0

6

7.

1

7

1

3	
4	

11

3

Tot

2- OPTIMAL PAGE REPLACEMENT ALGO (FUTURE)

This algorithm has lowest page fault rate for all algorithms of page replacement.

This algo states that "Replace that page which will not be used for longest period of time. i.e future knowledge of reference string is required.

⇒ lowest page fault

⇒ Replace the page that will not be used for the longest period of time

⇒ It is very difficult to implement.

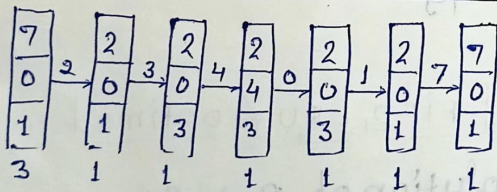
Prob:- calculate the no of page faults for the following reference string.

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

Soln let assume the No of frame = 3

Given reference string -

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1



Note victim page = RHS in reference string. जो page सबसे दूर था वो ही हटेगा। Replace कर देंगे।

Total No of page fault = 3 + 6
= 9.

Prob-1

Past LRU | Future OPT

future

Page Refs	3	2	1	3	4	1	6	2	4	3	4	2	1	4	5	2	1	3	4
f1	3	3	3	3	4	4	4	4	4	4	4	4	4	4	5	5	5	5	5
f2		2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	3	3
f3			1	1	1	1	6	6	6	3	3	3	1	1	1	1	1	1	4
Hit / Miss	X	X	X	✓	X	✓	X	✓	✓	X	✓	✓	X	✓	X	✓	✓	X	X

Replace a page that will not be used in near future when No element will be in future then follow FIFO

Total Page fault = 10

$$\text{Hit Ratio} = \frac{9}{19} \quad \left(\frac{\text{total No of Hit}}{\text{Total No of Element in ref str}} \right)$$

$$\text{Miss Ratio} = \frac{10}{19}$$

comparision of FIFO, LRU & optimal

According to the solution of Prob-2

Algo	Page fault	Hit Ratio
FIFO	13	6/19
LRU	14	5/19
optimal	10	9/19

THRASHING.

The phenomenon of moving pages from Primary to secondary storage or vice-versa consumes a lot of computer's energy but accomplishes very little useful results. This situation is called thrashing. A process is in thrashing if it is spending more time in paging instead of their execution.

As we know that the operating system always monitors CPU utilization. If CPU utilization is too low, we increase the degree of multiprogramming by introducing a new process into the system.

The CPU scheduler sees the decreasing CPU utilization and increases the degree of multiprogramming as a result. The new process tries to get started by taking frames from running processes causing more page fault. As a result CPU utilization again degrades. Seeing this again CPU scheduler increases the degree of multiprogramming, so now the situation comes where thrashing will take place and CPU utilization is dropped. The page fault rate is increased tremendously.

